

Command Design Pattern

Intent

Turn a **request into an object**. Instead of calling a method directly, you wrap "the thing to do" in a command object that knows *which* receiver to act on and *what* to do to it. Because the request is now an object, you can pass it around, store it, queue it, log it, undo it, or hand it to something that fires it later without knowing what it does.

Here the requests are **remote-control actions** (turn on, turn off, adjust volume, change channel), the things being controlled are **devices** (a TV, a Stereo), and the **RemoteControl** presses a button without knowing anything about the device behind it.

UML class diagram

```
<<interface>> Command <<interface>> Device | +execute() | | +turnOn() +turnOff() | ^ ^ ^ ^  
^ ^ | | | | | TurnOn TurnOff AdjustVolume ChangeChannel TV Stereo (Device) (Device)  
(Stereo) (TV) +changeChannel +adjustVolume \ | | / bound receiver + action  
+-----+ | RemoteControl | (invoker) | -command | | +setCommand() | |  
+pressButton() |--> command.execute() --> receiver method +-----+
```

The four roles (this pattern is defined by them)

```
controller/Command the COMMAND – the request interface: execute()  
controller/concrets/TurnOnCommand CONCRETE COMMANDS – each binds a receiver + an action  
/TurnOffCommand /AdjustVolumeCommand /ChangeChannelCommand receiver/Device (+ concrets/TV,  
Stereo) the RECEIVER – the object that does the real work invoker/RemoteControl the  
INVOKER – holds a command and triggers it CommandDesignPattern (main) the CLIENT – wires  
receivers, commands, invoker
```

- **Command** — a common interface with a single `execute()`.
 - **Concrete Command** — a small class that remembers *which receiver* and *which method* to call, and does so in `execute()`.
 - **Receiver** — the object that actually knows how to perform the work (the TV, the stereo).
 - **Invoker** — triggers a command (`pressButton()`) without knowing what the command does or who the receiver is.
 - **Client** — creates the receivers and commands and assigns commands to the invoker.
-

The code, line by line

Command — the request interface

```
public interface Command { public void execute(); }
```

- One method: `execute()` — "do the thing." Every request in this system is represented as an object implementing this interface.
- This is what makes a request first-class: anything that can hold a `Command` can trigger *any* action uniformly, without a giant `switch` over action types.

Device, TV, Stereo — the receivers

```
public interface Device { void turnOn(); void turnOff(); } public class TV implements Device { @Override public void turnOn() { System.out.println("TV is now on"); } @Override public void turnOff() { System.out.println("TV is now off"); } public void changeChannel() { System.out.println("Channel changed"); } } public class Stereo implements Device { @Override public void turnOn() { System.out.println("Stereo is now on"); } @Override public void turnOff() { System.out.println("Stereo is now off"); } public void adjustVolume() { System.out.println("Volume adjusted"); } }
```

- The **receivers contain the actual behavior** — they know how to turn on, change a channel, adjust volume. The commands don't *do* the work; they *delegate* to a receiver that does.
- `turnOn()/turnOff()` are shared across all devices, so they live in the **Device interface**. `changeChannel()` (TV-only) and `adjustVolume()` (Stereo-only) are device-specific, so they live only on the concrete classes. This split matters for how the commands are typed (below).

The concrete commands

```
public class TurnOnCommand implements Command { private Device device; public TurnOnCommand(Device device) { this.device = device; } @Override public void execute() { device.turnOn(); } } public class TurnOffCommand implements Command { private Device device; public TurnOffCommand(Device device) { this.device = device; } @Override public void execute() { device.turnOff(); } } public class AdjustVolumeCommand implements Command { private Stereo stereo; public AdjustVolumeCommand(Stereo stereo) { this.stereo = stereo; } @Override public void execute() { stereo.adjustVolume(); } } public class ChangeChannelCommand implements Command { private TV tv; public ChangeChannelCommand(TV tv) { this.tv = tv; } @Override public void execute() { tv.changeChannel(); } }
```

Each concrete command is the same tiny shape, and that shape *is* the pattern:

- **It stores a reference to its receiver** (a `Device`, `Stereo`, or `TV`) — this is the "who to act on," captured at construction time.

- Its `execute()` calls one method on that receiver — this is the "what to do." The command is essentially a bound (receiver, action) pair frozen into an object.
- Notice the receiver types differ on purpose:
- `TurnOnCommand`/`TurnOffCommand` take the **Device interface** — because every device can be turned on/off, so these commands work polymorphically with a TV, a Stereo, or any future device.
- `AdjustVolumeCommand` takes a **stereo** and `ChangeChannelCommand` takes a **tv** — because those actions exist only on those concrete receivers, so the command is typed to the receiver that actually offers the method.

RemoteControl — the invoker

```
public class RemoteControl { private Command command; public void setCommand(Command
command) { this.command = command; } public void pressButton() { if (command != null) {
command.execute(); } else { System.out.println("No command assigned"); } } }
```

- `setCommand(...)` — you load a command into the remote. The remote holds it **by the command interface**, so it can hold *any* command.
- `pressButton()` — the invoker's trigger. It just calls `command.execute()`. Critically, the remote has **no idea** whether it's turning on a TV or adjusting a stereo's volume — it only knows "I have a command; fire it." That ignorance is the entire benefit (see *Why* below).
- The `null` guard makes an unassigned button harmless ("No command assigned") instead of a `NullPointerException`.

CommandDesignPattern — the client

```
TV tv = new TV(); Stereo stereo = new Stereo(); Command turnOnTV = new TurnOnCommand(tv);
Command turnOffTV = new TurnOffCommand(tv); Command adjustVolume = new
AdjustVolumeCommand(stereo); Command changeChannel= new ChangeChannelCommand(tv);
RemoteControl remote = new RemoteControl(); remote.setCommand(turnOnTV);
remote.pressButton(); remote.setCommand(adjustVolume); remote.pressButton();
remote.setCommand(changeChannel);remote.pressButton(); remote.setCommand(turnOffTV);
remote.pressButton();
```

- The **client does the wiring**: it creates the receivers, builds commands that bind actions to those receivers, and assigns commands to the invoker.
- Then the **same** `pressButton()` produces four different behaviors, depending only on which command is currently loaded.

Console output:

Command Design Pattern TV is now on Volume adjusted Channel changed TV is now off

Why the design decisions

Why turn a method call into an object at all?

A direct call — `tv.turnOn()` — happens *right now* and can't be stored, passed, or reasoned about. Wrapping it as an object (`new TurnOnCommand(tv)`) makes the request something you can **hold and manipulate**: put it in a variable, store it in a list, queue it, log it, schedule it, or attach `undo()` to it later. Everything Command enables (undo/redo, macros, queues, transactional replay) flows from this one idea: *the request is now data*.

Why does the invoker hold a `Command` and not the receiver?

To **decouple the trigger from the work**. `RemoteControl` knows nothing about TVs or stereos — it depends only on the `Command` interface. That means:

- the same remote can trigger *any* action, present or future, and
- adding a new action (e.g. `MuteCommand`) requires **zero changes** to `RemoteControl`.

If the remote called `tv.turnOn()` directly, it would be welded to the TV and would need editing for every new device or action. Command breaks that coupling: the invoker fires requests without knowing what they are.

Why does each command store its receiver?

Because a command must know *what to act on* when it's eventually executed — possibly long after it was created, by an invoker that has no reference to the receiver. Binding the receiver into the command at construction time makes the command **self-contained**: `execute()` needs no arguments and no external context. That self-containment is what lets you queue or defer commands.

Why do the commands accept the receiver by *interface* where possible?

`TurnOnCommand(Device)` accepts the interface so one command class works for *every* device — turning on is universal, so there's no reason to tie the command to `TV`.

`AdjustVolumeCommand(Stereo)` accepts the concrete type only because `adjustVolume()` isn't part of `Device`. The rule: **depend on the narrowest type that still exposes the method you need** — the interface when the action is shared, the concrete class when the action is specific.

Why the `null` check in `pressButton()`?

Defensive robustness. An invoker can exist with no command loaded (a freshly-made remote, or a button that was never programmed). Handling that case gracefully ("No command assigned") is friendlier than crashing, and it documents that a command is optional state on the invoker.

Execution flow (the demo)

```
main (client wires everything) ■ ■■■■ remote.setCommand(turnOnTV) invoker now holds
TurnOnCommand(tv) ■■■■ remote.pressButton() ■ ■■■■ command.execute() →
TurnOnCommand.execute() → tv.turnOn() → "TV is now on" ■ ■■■■
remote.setCommand(adjustVolume) invoker now holds AdjustVolumeCommand(stereo) ■■■■
remote.pressButton() ■ ■■■■ command.execute() → AdjustVolumeCommand.execute() →
stereo.adjustVolume() → "Volume adjusted" ■ ■■■■ remote.setCommand(changeChannel) invoker
now holds ChangeChannelCommand(tv) ■■■■ remote.pressButton() ■ ■■■■ command.execute() →
ChangeChannelCommand.execute() → tv.changeChannel() → "Channel changed" ■ ■■■■
remote.setCommand(turnOffTV) invoker now holds TurnOffCommand(tv) remote.pressButton()
■■■■ command.execute() → TurnOffCommand.execute() → tv.turnOff() → "TV is now off"
```

The invoker does the identical thing every time (`command.execute()`); the varying behavior comes entirely from *which command object* is loaded — and the invoker never learns what any of them actually do.

Notes / possible extensions (not changed in the code)

- **Undo/redo.** The classic Command extension is adding `undo()` to the interface; each command remembers enough to reverse itself (often by holding a **Memento** of the receiver's prior state). A history stack of executed commands then gives multi-level undo.
 - **Macro commands.** A `MacroCommand` that holds a `List<Command>` and calls `execute()` on each lets you compose several requests into one — "movie mode" that dims lights, turns on the TV, and sets the stereo in a single button press.
 - **Queuing / logging / scheduling.** Because commands are objects, an invoker can push them onto a queue, run them on a worker thread, persist them to a log, or replay them — the basis of job queues and transactional systems.
 - **Lambdas.** `Command` is a single-method (functional) interface, so `remote.setCommand(tv::turnOn)` works identically — a method reference is a lightweight command.
 - **Plain main, no Spring** — the pattern is pure OO structure and needs no container.
-

Relationship to the other Behavioral patterns

- **Command (this module)** — package a request as an object so it can be stored, passed, queued, and undone; decouples the invoker from the receiver.
- **Memento** — commonly paired with Command to implement `undo()` (the command snapshots the receiver's prior state).
- **Strategy** — also wraps behavior in an object, but represents *how to do* one interchangeable algorithm, not *a request to be triggered/queued/undone* on a receiver.
- **Chain of Responsibility** — passes one request along handlers until one handles it; Command instead hands a fully-bound request to a single invoker to fire.

Reach for Command when you want to **parameterize objects with actions, queue or schedule** requests, support **undo/redo or macros**, or simply decouple the object that triggers an operation from the object that performs it.